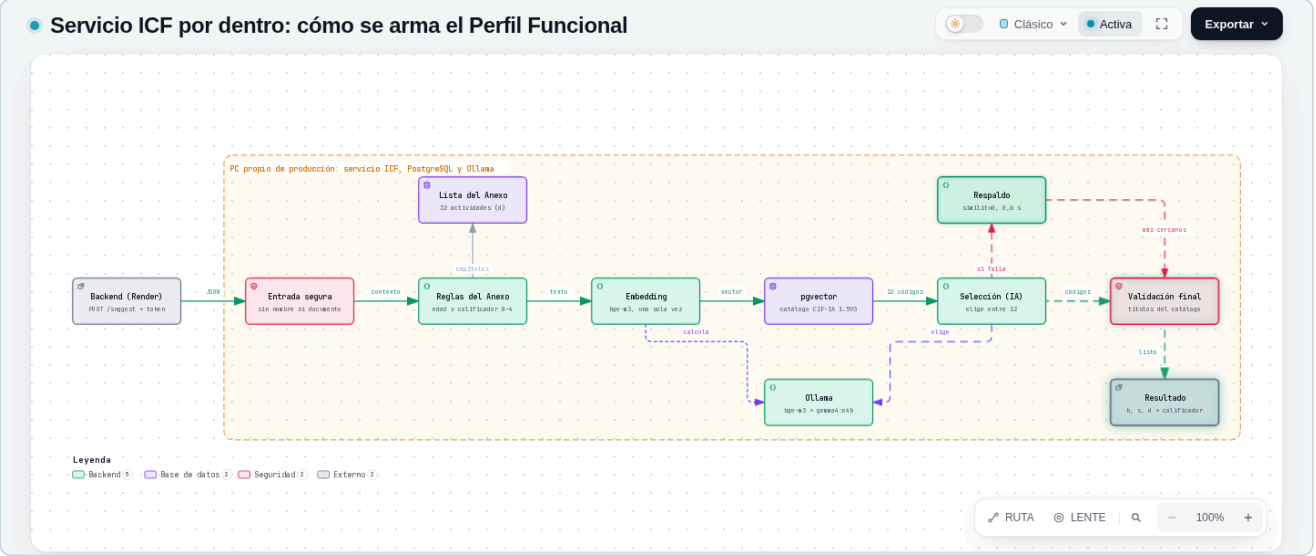
Manual del servicio ICF (perfil funcional con IA)

Health Access Bridge (HAB) · Versión 2.0 · 27 de octubre de 2026
Para: quien instala, opera y mantiene el servicio de IA que sugiere códigos CIF. No requiere programar, pero sí usar PowerShell y Docker con calma.
Complementa: Manual técnico · Manual de usuario · `icf-service/deploy/windows/` (archivos de despliegue) · Diseño del RAG.

Cómo funciona, de un vistazo



El backend envía un JSON sin nombre ni documento; las reglas del Anexo 1239 fijan edad y calificador; la búsqueda por significado trae 12 candidatos del catálogo; la IA solo elige entre ellos; todo se valida contra el catálogo y, si el modelo falla, entra el respaldo por similitud.
Versión interactiva y animada (Archify): abra `servicio-icf.html` en el navegador (clone el repositorio o descargue el archivo; GitHub muestra su código, no el diagrama). Para el contexto completo (Render, túnel, CI/CD) use `arquitectura-hab.html`.

Contenido

1. Qué es y qué no es
2. Arquitectura y flujo de una sugerencia
3. Cómo decide el motor

4. Requisitos de la máquina y rendimiento medido
5. Instalación desde cero
6. Configuración
7. Operación diaria
8. Mantenimiento
9. Seguridad y privacidad
10. Si algo falla
11. Pruebas y validación clínica
12. Licencias, límites y riesgos
13. Lista de comprobación rápida

---

1. Qué es y qué no es

Es un servicio que, dado el perfil de un paciente (edad, género, causa de la deficiencia, categorías, niveles D1–D6, predicción y —si el médico los escribe— diagnóstico y notas), **sugiere hasta tres códigos por componente** de la CIF-IA: funciones del cuerpo (**b**), estructuras del cuerpo (**s**) y actividades y participación (**d**), con calificador y justificación. El médico acepta, edita o rechaza cada uno desde la pantalla *Perfil Funcional ICF*.

No es un certificado de discapacidad ni un diagnóstico: es un **borrador de apoyo**. El certificado lo emite el equipo multidisciplinario en el RLCPD (Resolución 1239 de 2022).

Principios de diseño

- **Privacidad:** el modelo corre en un equipo propio; ningún dato clínico va a servicios de terceros.
- **Sin fine-tuning:** se usa un modelo abierto tal cual y se le acota el trabajo con búsqueda en el catálogo y reglas.
- **Verificable:** los títulos salen del catálogo y los calificadores de las reglas; el modelo solo **elige** entre candidatos del catálogo.
- **Degradación elegante:** si el modelo falla, el sistema devuelve la búsqueda por similitud; si el servicio falla del todo, la pantalla avisa y el resto de HAB sigue funcionando.

2. Arquitectura y flujo de una sugerencia



Componente	Función	Dónde corre
Ollama	Ejecuta los modelos con la GPU: `bge-m3` (embeddings) y `gemma4:e4b` (selección)	Instalado en Windows
Servicio ICF	Motor: reglas, búsqueda, selección y validación (`icf-service/app.py`)	Contenedor Docker
PostgreSQL + pgvector	Catálogo CIF-IA y sus embeddings. **No guarda pacientes**	Contenedor Docker
Caddy	Puerta de entrada: exige el token y solo deja pasar tres rutas	Contenedor Docker
Tailscale Funnel	Publica solo el puerto 11435 con HTTPS	Aplicación de Windows
Backend de HAB	Pide la sugerencia y **guarda** resultado y decisiones del médico	Render

El PC **no guarda datos de pacientes**: las sugerencias y decisiones viven en la base del backend. La base del PC (catálogo y embeddings) se puede reconstruir.

3. Cómo decide el motor

Para cada pedido a `POST /suggest` :

1. **Reglas fijas** (Anexo Técnico de la Resolución 1239 de 2022): según la edad y los niveles D1–D6 (capítulos con nivel ≥ 5) se determina qué dominios aplican, el calificador inicial (0–4) y los candidatos del Anexo. El perfil no aplica a menores de 6 años.
2. **Embedding del contexto clínico:** un solo embedding (`bge-m3`) del resumen del paciente, el diagnóstico y las notas.
3. **Actividades (d):** lista **cerrada** del Anexo, ordenada por calificador y por similitud. **No usa modelo de generación.**
4. **Funciones (b) y estructuras (s):** pgvector trae los candidatos más cercanos **entre códigos de tres dígitos** (12 en modo *calidad*, 6 en *rápido*; se excluyen los «otros» y «no especificados», terminados en 8 y 9). `gemma4:e4b` **elige y ordena** entre ellos con una salida estructurada (JSON

Schema, donde los candidatos son un conjunto cerrado) y, en modo calidad, una justificación corta por código.
5. **Validación:** todo código se comprueba contra el catálogo; el título sale del catálogo y el calificador de las reglas, **nunca del modelo**.
6. **Respaldo:** si el modelo no responde o devuelve algo inválido, se entregan los códigos más cercanos por similitud. Cada código lleva su **origen**: `llm` (modelo), `rules` (reglas) o `similarity` (respaldo).

Ajustes de la fase de validación clínica (HU-07g), activables con banderas (por defecto **activas**):

Bandera	Qué hace
`ICF_S_EVIDENCE`	Las **estructuras** (s) se sugieren solo si hay evidencia en el diagnóstico o las notas; evita completar la cuota de tres con estructuras sin sustento
`ICF_MOTOR_LINK`	Para cuadros motores, añade a los candidatos de **funciones** los códigos de fuerza, control del movimiento y marcha que la búsqueda por sí sola no traía
`ICF_MENTAL_LINK`	Vínculo equivalente para cuadros mentales. **Se recomienda apagarlo (`false`)**: en la ronda 3 contribuyó a 6 de 7 estructuras rechazadas

Modos (`ICF_MODE`): **calidad** (predeterminado; 12 candidatos, justificación del modelo) y **rápido** (6 candidatos, solo códigos). `ICF_USE_LLM=false` desactiva el modelo y deja solo similitud (~0,6 s).

4. Requisitos de la máquina y rendimiento medido

Recurso	Mínimo medido	PC de producción
Modelo de generación	`gemma4:e4b`	`gemma4:e4b`
GPU	NVIDIA con 8 GB (~3 s por sugerencia)	RTX 5050 de 8 GB
Solo CPU	Funciona, ~15 s por sugerencia	No aplica
RAM	16 GB	32 GB (Intel i5-13450HX)
Software	Windows, Docker Desktop, Ollama, Tailscale	Instalados
Red	Internet estable	—

Mediciones del 7-oct-2026 en la RTX 5050:

Situación	Tiempo por sugerencia
Modelo en memoria (uso normal)	~2,6 s
Primer pedido tras 30 min sin uso (carga del modelo)	~18 s
3 médicos a la vez	2,7 a 7,2 s
5 médicos a la vez	2,7 a 11,6 s

El backend espera hasta 90 s (`ICF_SUGGEST_TIMEOUT_S`). Informe completo: `PRUEBAS_HU07F_MODELOS_ABIERTOS.md` . El modelo anterior (`qwen2.5:3b` en un servidor sin GPU) se descartó: tardaba de 24 a 46 s y elegía peor que la similitud sola (`PRUEBAS_HU07_SERVIDOR_FISICO.md`).

5. Instalación desde cero

Se hizo el 7-oct-2026; estos pasos sirven para repetirla en otro PC.

Requisitos previos: Windows con sesión de usuario, GPU NVIDIA con controladores al día, Docker Desktop, Ollama, Tailscale con la sesión iniciada y el repositorio clonado. **Necesita además el archivo del catálogo CIF-IA** (`data\private\icf\catalog_cifia.tsv`), que tiene derechos de la OMS y **no está en el repositorio**: pídalo al responsable del proyecto.

1. **Modelos de Ollama**

   ```
   ollama pull gemma4:e4b
   ollama pull bge-m3
   ollama list          # deben aparecer ambos
   ```

2. **Secretos** (token y clave de la base; aleatorios, fuera del repositorio y de OneDrive)

   ```
   cd icf-service\deploy\windows
   .\init-env.ps1       # crea %USERPROFILE%\.hab-icf\.env; no muestra los valores
   ```

3. **Contenedores** (base, servicio y Caddy; se reinician solos)

   ```
   docker compose --env-file "$env:USERPROFILE\.hab-icf\.env" up -d --build
   ```

4. **Cargar el catálogo y los embeddings** (solo la primera vez o al reconstruir): ver la sección 8.3.
5. **Túnel público**

   ```
   & "C:\Program Files\Tailscale\tailscale.exe" funnel --bg 11435
   ```

   La URL pública es la del nodo (`tailscale status --json` , campo `Self.DNSName`), **sin barra final**.
6. **Verificar**

   ```
   .\check.ps1 -Public -Suggest      # debe terminar en "Todo correcto."
   ```

7. **Conectar con Render:** en los tres backends cargue `ICF_LLM_URL` (la URL pública), `ICF_LLM_TOKEN` (el mismo token del PC, que está en `%USERPROFILE%\.hab-icf\.env` como `ICF_TOKEN`) y `ICF_LLM_MODEL=gemma4:e4b` ; reinicie cada backend.
8. **Prueba final:** desde HAB como administrador, `GET /icf/health` debe responder que el modelo está disponible; y desde la pantalla *Perfil Funcional ICF*, genere un perfil de un paciente de prueba.

Si en el futuro se monta el servicio en un servidor **Linux**, la guía está en `icf-service/README.md` (docker-compose propio y `ollama` en el servidor).

6. Configuración

Dónde	Variable	Valor / uso
PC (`%USERPROFILE%\.hab-icf\.env`)	`ICF_TOKEN`	Token Bearer (lo crea `init-env.ps1`). **Nunca** en chats ni en el repositorio
PC	`ICF_DB_PASSWORD`	Clave de la base del catálogo (la crea `init-env.ps1`)
PC	`ICF_LLM_MODEL`	`gemma4:e4b`; cambiar de modelo no requiere tocar código
PC	`ICF_LLM_THINK`	`false` (Gemma 4 razona antes de responder; así se evita)
PC	`ICF_LLM_NUM_GPU`	Opcional; `0` fuerza CPU
PC	`ICF_MODE`	`calidad` (por defecto) o `rapido`
PC	`ICF_USE_LLM`	`true` por defecto; `false` = solo similitud
PC	`ICF_S_EVIDENCE`, `ICF_MOTOR_LINK`, `ICF_MENTAL_LINK`	Ver sección 3; recomendado `ICF_MENTAL_LINK=false`
PC	`ICF_LLM_KEEP_ALIVE`	Cuánto permanece el modelo en memoria (`30m`)
PC	`ICF_BODY_CANDIDATES`, `ICF_LLM_JUSTIFY`, `ICF_LLM_NUM_PREDICT`, `ICF_LLM_NUM_CTX`	Afinan el modo; los define `ICF_MODE`
Render (3 ambientes)	`ICF_LLM_URL`	URL pública del túnel, sin barra final
Render	`ICF_LLM_TOKEN`	El mismo token del PC
Render	`ICF_LLM_MODEL`	`gemma4:e4b`
Render	`ICF_LLM_TIMEOUT_S`, `ICF_SUGGEST_TIMEOUT_S`	Tiempos de espera; la sugerencia espera hasta 90 s

Después de cambiar una variable **del PC**: `docker compose --env-file "$env:USERPROFILE\.hab-icf\.env" up -d` (recrea lo necesario). Después de cambiar una variable **de Render**: reiniciar ese servicio.

7. Operación diaria

Todo corre cuando **inicia sesión el usuario de Windows**: Ollama y Docker Desktop arrancan con la sesión, los contenedores se reinician solos (`restart: unless-stopped`) y el túnel persiste. **Mantenga el cargador conectado**: con corriente alterna la suspensión y la hibernación están en «nunca»; con batería

el equipo se suspende a los 10 minutos. Revise en *Opciones de energía* que, «con corriente», cerrar la tapa diga «No hacer nada»; con la tapa cerrada el equipo necesita ventilación.

Qué	Comando (en `icf-service\deploy\windows`)
Comprobar todo (local)	`.\check.ps1`
Comprobar también la vía pública y una sugerencia	`.\check.ps1 -Public -Suggest`
Ver contenedores	`docker ps`
Ver registros del servicio	`docker logs icf-service --tail 50`
Reiniciar el servicio	`docker compose --env-file "$env:USERPROFILE\.hab-icf\.env" restart icf-service`
Ver el túnel	`tailscale funnel status`
Apagar el túnel (corta el acceso desde Render)	`tailscale funnel --https=443 off`
Volver a encenderlo	`tailscale funnel --bg 11435`
Ver desde HAB	`GET /icf/health` del backend (sesión de administrador)

> Si PowerShell no deja ejecutar el script: `powershell -ExecutionPolicy Bypass -File .\check.ps1 -Public -Suggest`.
> Si Docker Desktop está apagado, `docker` falla: ábralo y espere a que diga «running».

Después de reiniciar el PC: inicie sesión, espere uno o dos minutos y ejecute `.\check.ps1 -Public`. Si nadie inicia sesión, el servicio no está disponible y el backend usa el respaldo por similitud.

8. Mantenimiento

8.1 Rotar el token

1. `.\init-env.ps1 -Rotar` (genera token y clave nuevos; **no** cambia la clave que ya tiene la base).
2. Si rota la clave de la base, primero cámbiela dentro de la base: `"ALTER USER icf_user PASSWORD '<nueva>';" | docker exec -i icf-db psql -U icf_user -d icf`.
3. `docker compose --env-file "$env:USERPROFILE\.hab-icf\.env" up -d`.
4. Actualice `ICF_LLM_TOKEN` en Render (los tres ambientes) y reinicie cada servicio.
5. `.\check.ps1 -Public`.
   El token nunca se pega en chats, issues ni commits. Si se expone, se rota **de inmediato**.

8.2 Cambiar de modelo

1. `ollama pull <modelo>`.
2. Cambie `ICF_LLM_MODEL` en el `.env` del PC y en Render; recree los contenedores y reinicie Render.

3. **Verifique la licencia del modelo** (Gemma 4 es Apache 2.0; MedGemma **no** se usa: sus términos prohíben el uso clínico) y repita la validación clínica antes de usarlo con pacientes. Ver `LICENCIAS_COMPONENTES_ICF.md`.

8.3 Reconstruir la base (catálogo y embeddings)

Con los contenedores arriba y el catálogo en `data\private\icf\catalog_cifia.tsv`:

```
$envFile = "$env:USERPROFILE\.hab-icf\.env"
$pw = ((Get-Content $envFile | Where-Object { $_ -like "ICF_DB_PASSWORD=*" }) -split "=",2)[1]
cd <raíz del repositorio>
docker run --rm --network hab-icf_default `
  -e "ICF_DATABASE_URL=postgresql://icf_user:$pw@icf-db:5432/icf" -e
OLLAMA_URL=http://host.docker.internal:11434 `
  -v "${PWD}:/work" -w /work/icf-service icf-service python scripts/load_catalog.py
# repetir con scripts/embed_catalog.py (unos minutos; se puede reanudar)
```

`load_catalog.py --dry-run` valida el archivo sin escribir. Faltan 3 códigos sin definición en el libro (b1125, b239, d341); están fuera del catálogo.

8.4 Actualizar el servicio

Cuando cambie el código de `icf-service/`: `git pull`, `docker compose --env-file "$env:USERPROFILE\.hab-icf\.env" up -d --build` y `.\check.ps1 -Public -Suggest`.

8.5 Copias de seguridad

El PC no tiene datos de pacientes. Lo que conviene respaldar: el archivo `%USERPROFILE%\.hab-icf\.env` (en un lugar seguro, **no** en OneDrive ni en el repositorio) y el archivo del catálogo `catalog_cifia.tsv`.

9. Seguridad y privacidad

- **Qué viaja al servicio:** edad, género, causa, categorías, niveles D1–D6, la predicción y, si el médico los escribe, diagnóstico y notas. **Nunca** el nombre, el documento ni la orientación sexual. El servicio **no guarda** las notas.
- **Puerta:** Caddy exige el token Bearer en **todas** las rutas y solo deja pasar `/suggest`, `/health` y `/api/tags`; lo demás responde 404 (por ejemplo, `/api/generate` de Ollama no se expone).
- **Red:** Tailscale Funnel publica **solo** el puerto 11435; la base y el servicio no tienen puertos abiertos al exterior.
- **Secretos:** en `%USERPROFILE%\.hab-icf\.env`, fuera del repositorio y de OneDrive.
- **Notas libres:** pueden contener datos identificables si el médico los escribe; la pantalla lo advierte.
- **Marco legal:** Ley 1581 de 2012 (protección de datos personales). El perfil es un **borrador de apoyo**; la decisión es del profesional.
- **Auditoría:** cada sugerencia guarda el modelo que la generó, el lote y la decisión del médico (en el backend).

10. Si algo falla

Síntoma	Causa probable	Qué hacer
La pantalla dice que no se pudo generar el perfil	PC apagado, sin sesión, sin internet o túnel caído	Verificar que el PC esté encendido con la sesión iniciada; `.\check.ps1 -Public`; `tailscale funnel status`
Las sugerencias llegan con origen «Similitud» y no «Modelo»	Ollama o el modelo no responden y se usó el respaldo	`ollama list`; reiniciar Ollama; `.\check.ps1 -Suggest`
El primer pedido tarda ~20 s	El modelo se descargó de memoria tras 30 min sin uso	Normal; los siguientes tardan ~3 s
`/icf/health` da 401	Token distinto entre Render y el PC	Igualar `ICF_LLM_TOKEN` con el del PC y reiniciar Render
`/icf/health` da 403 desde la URL pública de Ollama	Ollama rechaza `Host` no local	Ya resuelto en el `Caddyfile` (`header_up Host`); si reaparece, revisar que Caddy esté arriba
`check.ps1` falla en la parte pública	Túnel apagado	`tailscale funnel --bg 11435`
Muy lento o sin GPU	Otra aplicación usa la GPU o falló el controlador	Cerrar aplicaciones pesadas; `nvidia-smi`; reiniciar Ollama
Sin respuesta tras reiniciar el PC	Falta iniciar sesión o Docker aún arranca	Iniciar sesión, esperar 1–2 minutos y `.\check.ps1 -Public`
La base está vacía o dañada	Contenedor recreado sin volumen	Reconstruir catálogo y embeddings (8.3)
Códigos sin sentido para el caso	Notas genéricas o ausentes	Pedir al médico diagnóstico y notas más específicos; revisar la validación (sección 11)
`docker` no responde	Docker Desktop apagado	Abrirlo y esperar a que arranque; `docker compose ... up -d`

11. Pruebas y validación clínica

Pruebas automáticas

Qué	Dónde	Cómo
Servicio ICF (67 pasan, 8 se omiten en el CI)	`icf-service/tests/`	`cd icf-service && pytest`
Backend de sugerencias	`backend/app/tests/` (`test_icf_*.py`)	`cd backend && pytest app/tests/`

Qué	Dónde	Cómo
Pantalla y guía	`frontend/src/__tests__/FunctionalProfile.test.tsx`, `IcfGuide.test.tsx`	`cd frontend && npm run test`
E2E del perfil (5 casos, servicio simulado)	`frontend/e2e/functional-profile.spec.ts`	`npx playwright test e2e/functional-profile.spec.ts`
Hoja de revisión clínica	`icf-service/scripts/review_sheet.py` (`generate`, `upgrade`, `score`)	Ver el protocolo

Las pruebas omitidas en el CI necesitan el catálogo CIF-IA (derechos de la OMS); se corren en el PC con `ICF_TEST_DATABASE_URL`.

Validación clínica (HU-07g). Una profesional de salud revisó el motor en **tres rondas** (24 casos sintéticos × 3, con una tercera ronda de **casos nuevos** y reglas fijadas por escrito antes de generar las sugerencias). Resultado final: **5 de 6 criterios cumplidos** sobre los casos nuevos; el que **no** cumple es la concordancia del calificador (κ ponderado 0,312 frente a la meta de 0,50). Por eso el sistema se presenta como **borrador de apoyo** y el calificador es un **valor inicial que el médico edita**; **no se aprueba el uso clínico autónomo**. Detalle y sustento de los umbrales: `PROTOCOLO_VALIDACION_CLINICA_HU07G.md` y reporte final del afinamiento.

Pendientes de la validación: segunda revisora, calificador en escala 0–4 con mejor ajuste, apagar `ICF_MENTAL_LINK` y una cuarta ronda con casos nuevos.

12. Licencias, límites y riesgos

- **Licencias** (verificadas el 7-oct-2026): Gemma 4 es Apache 2.0 (se puede usar); MedGemma no se usa. `bge-m3` se usa para embeddings. El **catálogo CIF-IA tiene derechos de la OMS**: no se redistribuye. Si se cambia el modelo, repetir la verificación.
- **Equipo personal sin redundancia:** puede apagarse, suspenderse o perder internet. Mitigaciones: respaldo por similitud en el backend, arranque automático y este manual.
- **Capacidad:** una sola máquina atiende a uno o pocos médicos a la vez (5 simultáneos: hasta 11,6 s).
- **Calidad:** el modelo puede equivocarse; el calificador concuerda solo parcialmente con el criterio clínico. Todo código debe ser validado por un profesional.
- **Consulta regulatoria** (si el software es o no dispositivo médico) y **permiso de uso de los títulos de la OMS**: se descartaron como actividades del proyecto; si el uso crece, conviene retomarlas.

13. Lista de comprobación rápida

Cada mañana (30 segundos)

- [ ] El PC está encendido, con la sesión iniciada y el cargador conectado.
- [ ] `.\check.ps1 -Public` termina en «Todo correcto.»

Después de un reinicio: iniciar sesión → esperar 1–2 min → `.\check.ps1 -Public`.
Si el médico reporta que no genera el perfil: `docker ps` → `tailscale funnel status` → `ollama list` → `.\check.ps1 -Public -Suggest`.
Si sospecha que se filtró el token: rotarlo ya (8.1).